PHY 115L
Introduction to the
Discrete Fourier Transform

Michael Mulhearn

September 29, 2025

1 Notation

We'll reserve k for the wave number of a particle. We will use m and n as dummy indices. So we will use M (capital) for the mass of a particle to avoid confusion with the index m. We'll use N for the number of samples. We'll use a for the period of a function. Note that the numpy FFT library uses different notation, taking k as the index for Fourier coefficients (our m) and n for the number of samples (our N).

If your HW solutions use the previous notation, that is fine, there is no need to update your notation!

2 Fourier Series for Complex-Valued Functions

We will treat the Fourier Series in the context of the inner product space of square-integrable periodic functions of period a, with an inner product defined as:

$$\langle f|g\rangle \coloneqq \int_{-\frac{a}{2}}^{\frac{a}{2}} f^*(x)g(x)dx$$

The complex exponential functions

$$e_m(x) \coloneqq \sqrt{\frac{1}{a}}\exp\left(i\frac{2\pi kx}{a}\right)$$

are orthonormal vectors. To verify:

$$\begin{aligned}\langle e_m|e_n\rangle &= \frac{1}{a}\int_{-\frac{a}{2}}^{\frac{a}{2}}\exp\left(i\frac{2\pi(n-m)x}{a}\right)dx \\ &= \frac{1}{\pi(n-m)}\cdot\frac{\exp\left(i\pi(n-m)\right)-\exp\left(-i\pi(n-m)\right)}{2i} \\ &= \mathrm{sinc}(\pi(n-m))\end{aligned}$$

As n and m are integers, and sinc is zero for every integer multiple of π except for $\mathrm{sinc}(0)=1$, we have shown:

$$\langle e_m|e_n\rangle = \delta_{mn} \tag{1}$$

We will not derive it here, but the complex functions are also a complete basis, so we can expand any function in terms of the basis of complex exponentials by:

$$\psi(x) = \sum_{m=-\infty}^{+\infty} c_m e_m(x) \tag{2}$$

Because of othonormality, we can determine the Fourier coefficients by:

$$c_m = \langle e_m | \psi \rangle \tag{3}$$

3 Discrete Fourier Transform

Next we want to consider the case where we only know the value of $\psi(x)$ at N discrete positions of x, specifically:

$$x_m = x_0 + m\,\Delta a$$

where

$$\Delta a \coloneqq \frac{a}{N}$$

for[1]

$$x_0 = -\frac{a}{2}$$

and

$$m = 0, 1, \ldots, N-1$$

Notice that **the sequence does not include a/2** because this point is redundant to $x_0 = -a/2$ for the periodic functions that we are considering.

We account for the discrete sampling by making the replacement:

$$\psi(x) \to \frac{a}{N} \sum_{m=0}^{N-1} \delta(x - x_m)\, \psi(x)$$

which is choosen because:

$$\int_{x_1}^{x_2} \frac{a}{N} \sum_{m=0}^{N-1} \delta(x - x_m)\, \psi(x)\, dx = \frac{a}{N} \sum_{m=m_1}^{m_2} \psi_m \xrightarrow[N\to\infty]{} \int_{x_1}^{x_2} \psi(x)\, dx$$

showing that integration (smoothing) of discrete sampled function reproduces the integral of the original function as the space between samples approaches zero. We do expect this sampling to introduce high-frequency artifacts which must be considered, which we will discuss later.

Calculating the Fourier series coefficients with the sampled function we obtain:

$$\begin{aligned} c_m &= \langle e_m | \psi \rangle \\ &= \sqrt{\frac{1}{a}} \int_{-\frac{a}{2}}^{\frac{a}{2}} \exp\left(-i\frac{2\pi m x}{a}\right) \frac{a}{N} \sum_{n=0}^{N-1} \delta(x - x_n)\, \psi(x)\, dx \\ &= \frac{\sqrt{a}}{N} \sum_{n=0}^{N-1} \exp\left(-i\frac{2\pi m n}{N}\right) \exp\left(-i\frac{2\pi n x_0}{a}\right) \psi_n\, dx \end{aligned}$$

[1] You might wonder why not just put $x_0 = 0$, and indeed, that would work fine for perioidic functions. However, choosing $x_0 = -a/2$ will be much more convenenient later when we are dealing with potentials centered at zero.

which we can reorganize as:

$$\frac{c_m}{\sqrt{a/N}} \exp\left(i\frac{2\pi m x_0}{a}\right) = \frac{1}{\sqrt{N}} \sum_{n=0}^{n-1} \exp\left(-i\frac{2\pi m n}{N}\right) \psi_m \tag{4}$$

Notice that the right hand side depends only on indices, not the actual x positions. This is an important insight. The FFT takes function values to coefficients, never knowing the actual x positions. If you checkout the documentation for `numpy.fft.fft`, you can confirm that the right hand side of that equation is Fourier coefficient A_m as provided by the tool with **the normalization option** `norm=''ortho''` So when moving between the coefficients provided by the tool and our derivations, there are some extra factors involved:

$$A_m \equiv \frac{c_m}{\sqrt{a/N}} \exp\left(i\frac{2\pi m x_0}{a}\right)$$

The complex exponential is a phase factor that accounts the location of the first data point x_0, which we are taking as $-a/2$. We can work with our c_m coefficients, but we'll have to account for these additional factors once we are using the library tools.

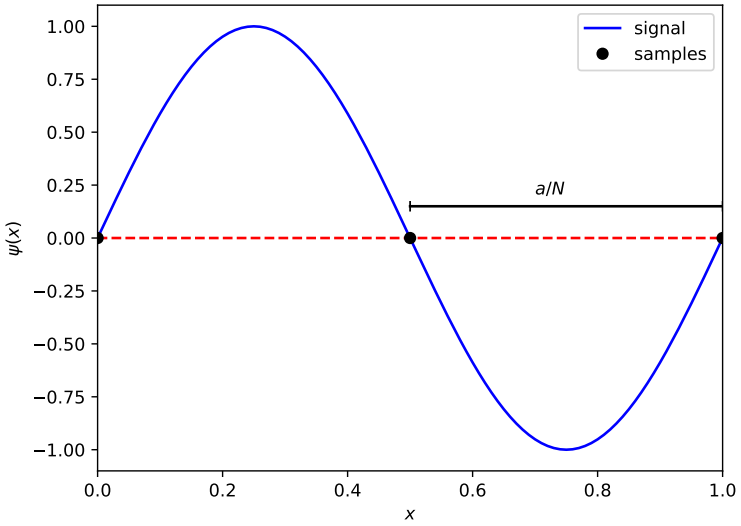

Figure 1: Trouble.

Something very interesting has happened to our Fourier coefficients as a direct result of discrete sampling. Have a look at Equation **??** and notice that:

$$c_{m+N} = c_m$$

because the phase shift introduced in each term in the sum above is:

$$\exp(-i2\pi N) = 1$$

So evidently we have only N unique Fourier coefficients, at which point they repeat themselves.

We can build an intuitive picture of what is going on from Fig. **??**. We see that signal contributions for wavelengths:

$$\lambda \leq 2\frac{a}{N}$$

are potentially problematic, as they can be undetectable. In terms of the index m in each c_m, this becomes:

$$\frac{a}{|m|} = \lambda < 2\frac{a}{N}$$

or

$$|m| < 2/N$$

For odd numbers of N this is an unambiguous prescription to avoid trouble:

$$\begin{aligned} N = 1 &\implies m = 0 \\ N = 3 &\implies m = -1, 0, 1 \\ N = 5 &\implies m = -2, -1, 0, 1, 2 \end{aligned}$$

We see that we have one unique coefficient available for each allowed wave number. We simply set the remaining coefficients to zero. If we know that our signal has no contributions from those shorter wavelengths then our Fourier series will be complete and accurate.

For even numbers of N the situation is a bit more complicated. If we stick to strictly less than for the allowed m values, we have:

$$\begin{aligned} N = 2 &\implies m = 0 \\ N = 4 &\implies m = -1, 0, 1 \\ N = 6 &\implies m = -2, -1, 0, 1, 2 \end{aligned}$$

which does not use all of our unique Fourier coefficients.

A completely reasonable and practical thing to do in the situation would be too say "Fine! Throw out the shorter wavelength coefficients. We'll just keep our sampling rate high enough that all of these coefficients are zero." This is reasonable, and totally works. The only downside is that it makes the DFT a lossy operation. We cannot get back the original sampled data back exactly from the transform. So it's worth tyring to fix things up. Look at what happens if we loosen the requirements to on m to less than or equal:

$$\begin{aligned} N = 2 &\implies m = -1, 0, 1 \\ N = 4 &\implies m = -2, -1, 0, 1, 2 \\ N = 6 &\implies m = -3, -2, -1, 0, 1, 2, 3 \end{aligned}$$

which would require one more Fourier coefficient than we have. What's going on?

As can be seen from Fig **??** some phases can be sampled, even at the limit, but not all. So we need to know something about the the contribution at the limit: if it's purely real and even, for example. In practice, we make the assumption that makes the inverse discrete Fourier transform the easiest to write down (just drop the $c_{n/2}$ factor), and make the allowed m values:

$$\begin{aligned} N = 2 &\implies m = -1, 0 \\ N = 4 &\implies m = -2, -1, 0, 1 \\ N = 6 &\implies m = -3, -2, -1, 0, 1, 2 \end{aligned}$$

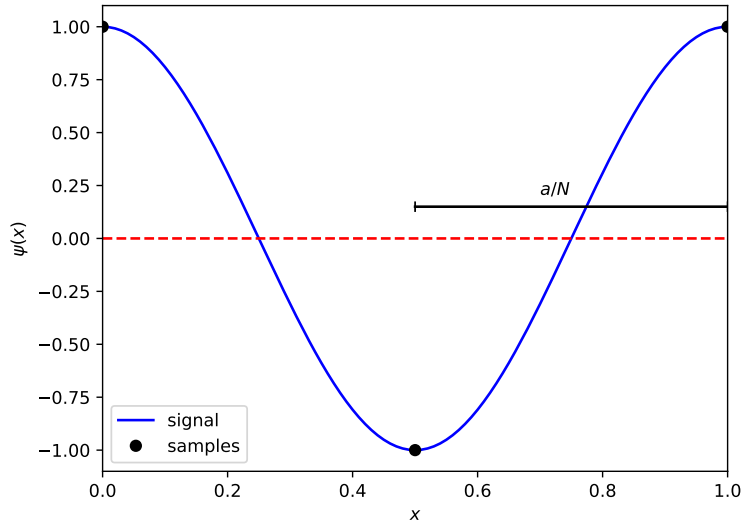

Figure 2: No trouble.

This puts one term in the series for each unique coefficient, and makes the DFT lossless.

This book keeping is conveniently handled for you by python with the `numpy.fft.fftfreq` function, which returns an array the same size as the Fourier coefficients that reports m/n. For equation purposes we will note the set of allowed m values as $M(N)$.

We can reconstitute our original wave function (wihtout any artifacts from sampling) as:

$$\begin{aligned}
\psi(x) &= \sum_{m \in M(N)} c_m e_m(x) \\
&= \frac{1}{\sqrt{a}} \sum_{m \in M(N)} c_m \exp\left(i \frac{2\pi k x}{a}\right) \\
&= \frac{1}{\sqrt{n}} \sum_{m \in M(N)} A_m \exp\left(i \frac{2\pi k (x - x_0)}{a}\right)
\end{aligned}$$

4 Homework Problems

Please submit both a PDF file (including output) and a python notebook file (.ipynb) of your completed assignment.

Problem 1: Use the `numpy.fft.fftfreq` (which produces an array with m/N for m with non-zero

coefficients) to reproduce the progression we derived above:

$$\begin{aligned}
N = 1 &\implies m = 0 \\
N = 2 &\implies m = -1, 0 \\
N = 3 &\implies m = -1, 0, 1 \\
N = 4 &\implies m = -2, -1, 0, 1 \\
N = 5 &\implies m = -2, -1, 0, 1, 2 \\
N = 6 &\implies m = -3, -2, -1, 0, 1, 2 \\
N = 7 &\implies m = -3, -2, -1, 0, 1, 2, 3
\end{aligned}$$

The `fftfreq` function returns an array with the elements in a different order, so also use `numpy.fft.fftshift` to put them into increasing order and scale as needed. **Important:** note that `fftfreq` and `fft` present results in the same order. So if you reorder the wavenumbers with `fftshift` you should appply the same shift to your coefficients.

Problem 2: Define:

$$\psi(x) = A + B\sin(2\pi bx/a) + C\cos(2\pi cx/a)$$

where a and b are integers and you can assume (to keep things simple) that $a \neq b$. You should show final plots for $a = 30$ and $n = 10$ but make sure your code runs even if you change those values modestly. Use `numpy.fft.fft` to calculate the fft of $\psi(x)$ using the `norm='ortho'` normalization option. Plot the magnitude, real part, and imaginary part of the coefficients (scaled as you feel appropriate) versus the wavenumber. Set the range appropriately. Draw horizontal and vertical line for the y-values and x-values that you expect for the magnitude of the cosine curve (make sure these are caculated from constants used to define your $\psi(x)$. (Hint: attack this and the next problem in steps: first get the constant function working alone, then sine alone, then cosine alone, then put it all together.)

Problem 3: Using the results from your previous problem, plot the the function $\psi(x)$ caculated from the Fourier coefficients and compare to the original function. Use much finer binning (200 bins) and a wider x range (say -40 to 40) to generate smooth curves (without changing the sampling rate used for the FFT in previous step).

Problem 4: Demonstrate that you can obtain Fourier coefficients and accurately reproduce a periodic square wave (sample for one period, but plot for several periods, and adjust parameters as needed for accurate results)

Problem 5: Demonstrate that you can obtain Fourier coefficients and accurately reproduce a narrow square well near $x = 0$. (The square well width should be small compared to the period)

Problem 6: Show that a sine wave with frequency larger than one half the sample frequency is misintrepreted as a lower frequency sine wave. This is called aliasing. Plot the signal and (incorrect) reconstructed wave form. Include a simple calculation for the aliased wavenumber if you can discern a formula.